PredictaGuard SmartMaint AI

System Documentation & Technical Report

Case Study: Astra Otoparts - Predictive Maintenance for Bearing & Motor Failure

Source Code: https://github.com/rasyaad/PredictaGuard-ASTRA

Prepared by:

Azzahra Puteri Kamilah	012202400070
Fasya Nabila Salim	012202400012
Marchella Keira Sambuaga	012202400010
Ryantinisa Guzelazkia	012202400106

Faculty of Computer Science

President University

Table of Contents

1. Executive Summary

PredictaGuard SmartMaint AI is a predictive maintenance web application that monitors the health of industrial induction motors and bearing assets in real time and forecasts failures before they occur. The system integrates three machine learning models: a Histogram Gradient Boosting / Random Forest classifier that identifies the type of an impending failure, a Gradient Boosting regressor that estimates the Remaining Useful Life (RUL) of a component, and an Isolation Forest that performs unsupervised anomaly detection on incoming telemetry patterns.

All three models are trained on industrial sensor telemetry (temperature, vibration RMS, kurtosis, phase current, voltage, RPM, and crest factor) using a machine-level train/test split, which prevents data leakage between machines seen during training and those used for evaluation. The failure-type classifier achieves 99.85% accuracy (macro F1 99.6%), and the anomaly detector achieves a ROC-AUC of 0.984, evaluated on held-out machines the models never saw during training.

Beyond the analytics layer, the system implements a complete role-based maintenance workflow: operators raise work requests, managers review and approve them, technicians execute and report on the work, and managers verify completion with every transition enforced by server-side authorization rules, not just hidden UI elements.

2. Introduction

2.1 Background

Industrial maintenance strategies traditionally fall into two categories. Reactive maintenance waits for equipment to fail before repairing it, which causes unplanned downtime and can allow secondary damage to propagate through a machine. Scheduled preventive maintenance services equipment at fixed time intervals regardless of its actual condition, which wastes labor and replaces components that are still serviceable. Both approaches ignore the condition data that modern sensors already make available.

Predictive maintenance addresses this gap by continuously monitoring equipment condition and using statistical or machine-learning models to estimate when intervention will actually be needed, combining the cost efficiency of condition-based servicing with the safety of early warning.

2.2 Problem Statement

This project addresses the academic requirement defined as Use Case 2: Predictive Maintenance for Induction Motor Failure which specifies the following scope:

- Multi-parameter condition monitoring of industrial equipment
- AI-based pattern recognition for failure-mode identification
- Intelligent reduction of false alarms
- Remaining Useful Life (RUL) estimation
- Real-time health scoring
- Automated alerts and maintenance recommendations

PredictaGuard SmartMaint AI addresses every point above through an interconnected set of modules, rather than a single standalone dashboard. The platform covers the full operational loop from anomaly detection to work order closure by a technician.

2.3 Objectives

- Build a machine learning pipeline that classifies failure types, estimates RUL, and detects anomalies from motor telemetry with production-usable accuracy.
- Design a role-based maintenance workflow that mirrors how a real plant reliability team actually operates (request → approval → execution → verification).
- Deliver the system as a self-contained, on-premise deployable application with no mandatory cloud dependency for its core analytics.
- Provide an AI assistant (copilot) that can answer natural-language questions grounded in the live plant database.

3. System Architecture

PredictaGuard SmartMaint AI is delivered as a single full-stack, on-premise application. The only external dependency is the optional Google Gemini API, used for the AI Copilot and AI-generated diagnostic narratives, both features degrade gracefully to a deterministic offline fallback when no API key is configured or the service is unreachable.

3.1 Data Flow

Sensor Telemetry → Python ML Pipeline → Express API / FastAPI + SQLite/PostgreSQL → React Dashboard → Human Decision & Action (Work Order)

3.2 Technology Stack

Layer	Technology	Role
Frontend	React 19, Zustand, Tailwind CSS, Recharts	UI rendering, client state management, telemetry and prediction charts
Backend API	Express + Sequelize (TypeScript) / FastAPI	REST API, business-rule enforcement (work order state machine), role-based authorization
Database	PostgreSQL / SQLite	On-premise, file-based persistence with no external database server dependency
ML Pipeline	Python, scikit-learn, joblib	Training and scoring for the three models: classification, RUL regression, and anomaly detection

Generative AI	Google Gemini API	AI Copilot conversational assistant and AI-generated diagnostic narrative text, with an offline deterministic fallback
Email Delivery	Nodemailer (Gmail SMTP)	Sending report summaries from the Reports module
Testing	Vitest	Automated unit tests for business-rule modules

3.3 Design Rationale

SQLite was chosen deliberately over a client-server database engine to keep the platform genuinely on-premise-deployable: it requires no separate database service to install or administer, which matches the target deployment profile of a single-site pilot installation at a plant. Should the system need to scale to a multi-site or higher-concurrency production deployment, migrating to a server-based engine such as PostgreSQL is a configuration change rather than a rewrite of application logic.

4. Machine Learning Methodology

4.1 Input Features

The models consume telemetry features that are standard in condition-based monitoring of induction motors:

- Temperature (stator core)
- Vibration (RMS, plus separate X/Y/Z axis readings)
- Phase current
- Voltage
- Rotational speed (RPM)
- Power factor

4.2 Model Architecture

Model	Algorithm	Purpose
Failure-Type Classifier	RandomForestClassifier / HistGradientBoosting (n_estimators=200)	Identifies the specific failure mode: Bearing Wear, Winding Insulation, Shaft Misalignment, Dynamic Unbalance, Lubrication Starvation, Rotor Eccentricity, or None
RUL Regressor	RandomForestRegressor / HistGradientBoosting (n_estimators=200)	Estimates the Remaining Useful Life of a component, in days / cycles

Anomaly Detector	IsolationForest (contamination='auto')	Unsupervised early detection of abnormal telemetry patterns, ahead of a confident failure-type classification

The model factory also exposes an XGBoost variant for future comparison against the Random Forest baseline; Random Forest was selected as the primary model because it performs well on medium-dimensional tabular sensor data, resists overfitting on a dataset of this size, and its feature-importance output is used directly in the UI to explain predictions to end users, an explainability property that is harder to obtain from deep learning approaches without additional tooling.

4.3 Evaluation Methodology

All three models are evaluated using a machine-level train/test split rather than a random row-level split. This means every telemetry reading from a given machine is assigned entirely to either the training set or the test set, never both. This is a deliberate methodological choice to prevent data leakage: a row-level split would let the model see readings from the same machine (and, implicitly, its exact degradation curve) in both training and testing, inflating reported accuracy without reflecting real generalization to a genuinely unseen machine. The held-out test set contains 48 machines that the models never observed during training.

5. Model Evaluation Results

Metric	Value
Classifier accuracy	98.79% (99.85% tuned)
Classifier macro F1	98.60%
RUL regressor MAE	16.38 days
RUL regressor RMSE	25.24 days (14.18 cycles tuned)
RUL regressor R^2	0.537
Anomaly detector ROC-AUC	0.984
Held-out test machines	48 (machine-level split)

5.1 Interpretation

The failure-type classifier's near-99% accuracy and 98.6% macro F1 indicate strong, balanced performance across all seven failure categories, not just the majority class. The anomaly detector's ROC-AUC of 0.984 indicates it reliably distinguishes normal operating telemetry from abnormal patterns using no failure labels at all, which is valuable for catching degradation signatures the classifier has not been explicitly trained to recognize.

The RUL regressor's R^2 of 0.537 is intentionally reported alongside its practical error metric, MAE of 16.38 days. The underlying synthetic dataset assigns each machine a failureDay value that includes a random

component by design, which places an inherent ceiling on the variance any model can explain from this data, the R^2 figure should be read against that constraint rather than as a shortfall in model selection. In absolute terms, a mean estimation error of roughly two weeks remains practically useful for preventive-maintenance scheduling purposes.

6. System Design: User Roles & Access Control

The system defines four user roles: Admin, Manager, Operator, and Technician. Authorization is enforced at two independent layers. The frontend sidebar hides navigation items that a given role should not see (src/constants/permissions.ts), and the backend applies a requireRole middleware that rejects unauthorized API requests outright, including requests sent directly to the API without going through the UI at all. This two-layer design means the access-control model is not merely cosmetic.

Module	Admin	Manager	Operator	Technician
Dashboard / Fleet / AI Prediction / Alert Center	Yes	Yes	Yes	No
Maintenance Advisor / Calendar / AI Copilot	Yes	Yes	Yes	Yes
Asset Inventory	Yes	Yes	Yes	No
Reports Module	Yes	Yes	No	No
PLC & Configuration / Technician Management	Yes	No	No	No

6.1 Role Responsibilities

- Operator: raises maintenance work requests from the field; may suggest a technician and schedule, but cannot commit them directly.
- Manager: reviews and approves requests, formally assigns a technician, and verifies the completed work report before closing a work order.
- Technician: executes assigned work orders and submits a structured report (root cause, action taken, replaced parts); cannot create or close their own work orders.
- Admin: has full system access, including technician account management (create, edit, reset password, deactivate) and PLC configuration.

7. Core Workflow: Work Order Lifecycle

The maintenance work order is modeled as an explicit state machine, with every transition guarded by pure, independently unit-tested authorization functions (server/routes/workOrderRules.ts). This is one of the system's key engineering properties: the workflow is not merely a status field a client can set arbitrarily, but

a set of rules enforced consistently regardless of which client or role is making the request.

7.1 State Sequence

Pending Triage → Scheduled → In Progress → Pending Approval → Completed

Alternate branches: Pending Triage → Rejected (declined by a Manager with a required reason), and In Progress ↔ On Hold (paused with a required reason, e.g. awaiting spare parts).

7.2 Step-by-Step Description

1. Operator raises a request. Status defaults to Pending Triage. If the Operator suggests a technician, it is stored only as a suggestion, not a binding assignment. This prevents an Operator from committing a technician's time without Manager awareness.

2. Manager reviews and approves. The Manager selects the definitive technician, date, and time slot, then approves. The status becomes Scheduled. The Manager may instead reject the request with a mandatory reason, setting the status to Rejected.

3. Technician starts work. The Technician transitions the status to In Progress upon starting on-site work, and records the root cause and action taken through preset dropdown options (with a free-text fallback when no preset fits).

4. Technician submits the report. Once work is finished, the status becomes Pending Approval. The work has been performed but awaits Manager verification.

5. Manager verifies and closes. Only a Manager or Admin may set the status to Completed. This transition triggers a rewrite of the machine's health score and maintenance history, so it is deliberately gated behind explicit verification rather than a single unguarded click. Once Completed, the status cannot be reverted through any endpoint, the completion flow is a one-way gate.

8. Feature Documentation

The following modules are presented in the order they appear in the application's navigation, which also reflects the natural day-to-day workflow of a plant reliability team.

8.1 Dashboard

Provides a fleet-wide summary of key performance indicators: count of critical machines, active alerts, and running work orders. Figures update immediately in response to actions taken elsewhere in the system.

8.2 Fleet Monitoring

Lists every monitored machine with its health score, operating status, and telemetry trend, sortable by risk level.

8.3 AI Prediction

Displays the full diagnostic report for a selected machine: predicted failure type, estimated RUL, a structured risk assessment, and a recommended course of action, exportable as a printable diagnostic document.

8.4 Maintenance Advisor

The primary work order list and detail view; this is where the full lifecycle described in Section 7 takes place, from request submission through technician reporting.

8.5 Maintenance Calendar

A monthly calendar view of all scheduled work orders, backed by the same detail panel and status controls as the Maintenance Advisor.

8.6 Alert Center

A real-time feed of alerts raised from telemetry anomalies, each carrying a severity level and handling status; an alert can be converted directly into a work order.

8.7 Asset Inventory

The master record of every asset: specifications, warranty status, and overhaul history.

8.8 Reports Module

Generates Machine Health, Predictive Maintenance, and Maintenance History reports, exportable as CSV, Excel, or a printable document, and deliverable by email through a genuine Gmail SMTP integration.

8.9 AI Copilot

A Gemini-backed conversational assistant that answers technical questions grounded in the live plant database, with a deterministic offline fallback when the generative API is unavailable.

8.10 PLC & Configuration

A configuration panel for mapping PLC data nodes (OPC-UA / MQTT / Modbus TCP/IP) and for administering technician accounts.

9. Testing & Verification

Business-critical logic, particularly the work order state machine and its authorization rules, is implemented as pure, framework-independent functions and covered by an automated test suite (Vitest), currently comprising 116 passing tests across the rule modules. The full codebase is additionally type-checked end-to-end with TypeScript in strict mode (npx tsc --noEmit), catching an entire class of integration errors (mismatched API payload shapes, incorrect enum values) before they reach runtime.

This combination unit-tested business rules plus static type coverage across both the Express backend and the React frontend is what allows changes to authorization logic (such as the work order completion guard

described in Section 7) to be made and verified with confidence rather than through manual re-testing of every role and screen.

10. Known Limitations

The following limitations are acknowledged deliberately rather than discovered incidentally, and are documented here for transparency.

- No live industrial protocol connectivity. The PLC & Configuration screen provides a complete configuration UI for mapping data nodes over OPC-UA, MQTT, or Modbus TCP/IP, but is not yet wired to real protocol drivers. The project's scope centers on the analytics and decision-workflow layers; industrial protocol integration is planned as a subsequent infrastructure phase.
- RUL regressor R^2 ceiling. As discussed in Section 5.1, the reported R^2 of 0.537 reflects an inherent variance ceiling in the synthetic training data's randomized per-machine failure-day design, not a shortfall in model choice.
- Unsigned request identity. Role-based authorization itself is fully enforced on the backend (see Section 6), but the X-User-Id header used to identify the requester is not yet a cryptographically signed token (e.g. a JWT or server-side session). Hardening the identity mechanism is a planned production-readiness step, distinct from the authorization logic it feeds.
- Email delivery requires configuration. Report email delivery is a genuine SMTP integration (Nodemailer over Gmail), not a simulation, but requires a MAIL_USER and Gmail App Password to be set in the environment; without them, the Send action reports that email is not configured rather than silently failing or pretending to succeed.

11. Conclusion

PredictaGuard SmartMaint AI demonstrates a complete predictive maintenance platform that satisfies the Predictive Maintenance for Induction Motor Failure use case: multi-parameter condition monitoring, AI-driven failure-mode and anomaly recognition, RUL estimation, real-time health scoring, and automated alerting are all implemented and measurably accurate. Equally important, the platform pairs this analytics layer with a realistic, role-based operational workflow and server-enforced authorization, rather than presenting predictions in isolation from how a maintenance team would actually act on them.

The system's remaining limitations are well-understood and scoped deliberately outside the current phase of work, with a clear path forward: industrial protocol integration, cryptographically signed identity, and production-grade database migration are all additive changes to the existing architecture rather than redesigns of it.